

VR Teleoperation & Inverse Kinematics

Architecture of the **vr_teleop** package in the Daksha workspace

Covers: the ROS 2 nodes that make up the Meta Quest → Daksha teleoperation bridge, how controller poses become an inverse-kinematics (IK) target and are solved with Placo, and how the trigger buttons drive the two grippers.

Package	vr_teleop (+ kinematics, gen2 for downstream context)
Workspace	/home/s1/.ihub/.final_gen2 — Daksha dual-arm robot
ROS distro	ROS 2 Humble
Robot description	daksha_description_full_body (robot.urdf)

Generated from the source in `src/tele/vr_teleop` and its two downstream consumers, `src/controllers/kinematics` and `src/gen2`.

1. Overview

vr_teleop is the bridge between a Meta Quest headset running a Unity app and the rest of the Daksha ROS 2 stack. The headset streams head and controller poses (plus trigger/grip button state) over a raw TCP socket using Unity's ROS-TCP-Connector wire protocol. **vr_teleop** terminates that socket, converts the poses from Unity's left-handed axis convention into ROS's right-handed FLU convention, and republishes everything as ordinary ROS topics and TF frames.

vr_teleop itself does **not** compute any robot joint angles. It only produces two clean per-arm target poses (`/left/pose`, `/right/pose`) and raw joystick/trigger data (`/quest/left/joy`, `/quest/right/joy`). Two separate consumers turn that into motion:

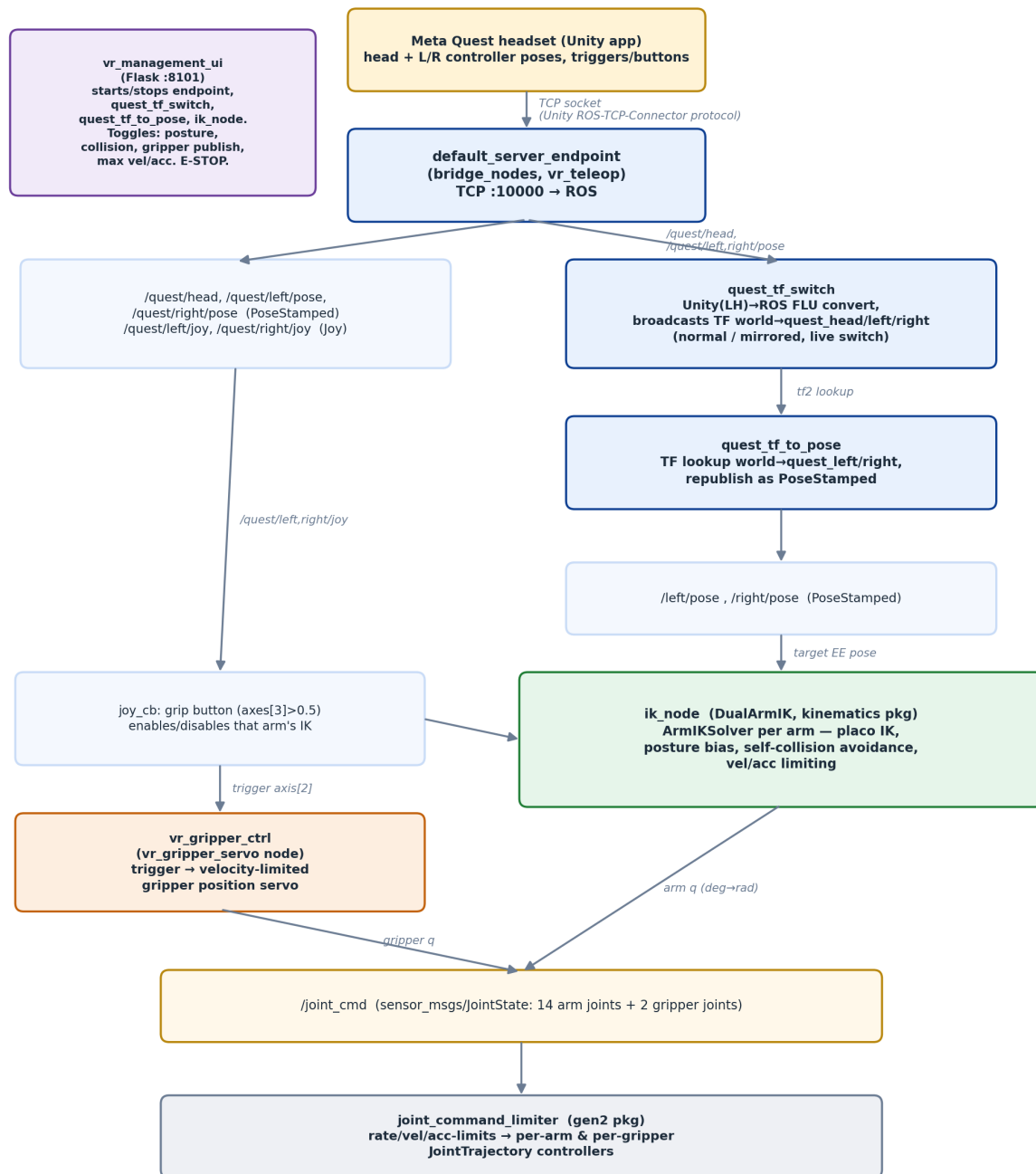
- **ik_node** (package `kinematics`) — solves arm joint angles from the target pose with a Placo-based IK solver.
- **vr_gripper_ctrl** (in **vr_teleop** itself) — turns the trigger squeeze into a gripper open/close position.

Both publish joint targets onto the shared `/joint_cmd` topic, which a downstream node (`joint_command_limiter`, package `gen2`) rate/velocity/acceleration-limits before handing it to the arm and gripper trajectory controllers.

A supervisor node, **vr_management_ui**, starts and stops the whole chain and exposes runtime toggles (posture bias, self-collision avoidance, gripper publishing, velocity/acceleration limits) plus an emergency stop, through a small web page on port 8101.

2. End-to-end data flow

vr_teleop → kinematics data flow (Daksha)



Two parallel paths leave the TCP endpoint: the **pose path** (right side of the diagram) carries controller position/orientation through TF and into IK; the **button path** (left side) carries the grip button (enable/disable IK) and the trigger (gripper open/close). Both converge on **/joint_cmd**.

3. Nodes at a glance

vr_teleop package (this package)

Node / executable	Role	Key topics / services
default_server_endpoint	Unity ROS-TCP-Endpoint server. Terminates the Quest's TCP connection (default 0.0.0.0:10000), decodes the wire protocol, and publishes/subscribes ROS messages on the Quest app's behalf.	Publishes <code>/quest/head</code> , <code>/quest/left/pose</code> , <code>/quest/right/pose</code> (PoseStamped) and <code>/quest/left/joy</code> , <code>/quest/right/joy</code> (Joy).
quest_tf_switch (recommended)	Converts each incoming PoseStamped from Unity LH axes to ROS FLU and broadcasts it as a TF transform. Supports switching between normal and mirrored mapping at runtime, with the head's yaw exempted from mirroring by default.	TF: <code>world→quest_head/left/right</code> . Service <code>~/set_mirror</code> (std_srvs/SetBool). Latched topic <code>~/mirror_state</code> .
quest_tf / quest_tf_mirror	Legacy fixed-mode predecessors of quest_tf_switch (normal-only / mirrored-only). Kept for old launch files; behave identically to the corresponding switch mode.	Same TF frames as quest_tf_switch.
quest_tf_to_pose	Looks up the <code>world→quest_left/right</code> TF transforms on a 100 Hz timer and republishes them as PoseStamped, so ik_node can consume plain topics instead of doing TF lookups itself. Waits for a live Quest connection before it starts looking anything up.	Subscribes TF. Publishes <code>/left/pose</code> , <code>/right/pose</code> (PoseStamped) – the exact topics ik_node subscribes to.
vr_pose_relay (alternate/legacy path)	A simpler, TF-free alternative to quest_tf_switch + quest_tf_to_pose: time-synchronizes the two raw Quest pose topics and relays them straight through to <code>/left,right/pose</code> , gated by an enable service. Not used by the default launch files.	Service <code>/vr_enable</code> (SetBool).
vr_gripper_ctrl	Reads each controller's analog trigger and servos a virtual gripper position toward open/closed at a velocity-limited rate; see §5 for the control law.	Subscribes <code>/quest/left,right/joy</code> . Publishes <code>/joint_cmd</code> (JointState) when enabled.
vr_management_ui	Process supervisor + Flask dashboard (port 8101). Starts/stops the endpoint, quest_tf_switch, quest_tf_to_pose and (5 s later) ik_node as one group; streams their stdout; exposes toggles and an E-STOP.	Parameter enabled. Web UI at <code>:8101</code> .

Downstream, outside this package

Node	Package	Role in the VR pipeline
ik_node (class DualArmIK)	kinematics	Subscribes to <code>/left,right/pose</code> and <code>/quest/left,right/joy</code> ; runs one ArmIKSolver per arm; publishes joint angles. Full detail in §4.
joint_command_limiter	gen2 (joint_cmd.py)	Consumes <code>/joint_cmd</code> from both ik_node and vr_gripper_ctrl, applies rate/velocity/acceleration limits, and issues JointTrajectory goals to the arm and gripper controllers. The second finger of each gripper is a URDF mimic joint, so only one joint per gripper is actually commanded.

4. Coordinate conversion: normal vs. mirrored

Unity uses a left-handed axis convention (X right, Y up, Z forward); ROS uses right-handed FLU (X forward, Y left, Z up). Converting between them is not a simple relabeling once orientation is involved — swapping axes on a rotation flips its handedness, so the quaternion's w component must also be negated to keep the rotation direction correct. `quest_tf_switch` applies this exactly once, in `broadcast()`:

Normal (direct mapping)	Mirrored (reflected + hands swapped)
<pre>pos = (uz, -ux, uy) rot = (uz, -ux, uy, -uw)</pre>	<pre>pos = (uz, ux, uy) rot = (uz, ux, uy, uw)</pre>

Normal mode treats the operator as sitting inside the robot: the left hand drives the `quest_left` frame (and therefore the left arm). **Mirrored** mode reflects positions through the plane normal to ROS Y and swaps which controller drives which TF frame, so the robot moves like the operator's reflection — intuitive when standing in front of the robot facing it. One deliberate exception: the head's yaw is *not* mirrored by default (`mirror_head_yaw` parameter), because a reversed gaze direction while looking through the headset is disorienting; position is still reflected.

The mode can be flipped live without restarting anything: `ros2 service call /quest_tf_switch/set_mirror std_srvs/srv/SetBool "{data: true}"`, and the active mode is also exposed on the latched topic `/quest_tf_switch/mirror_state` for other nodes to pick up.

5. How the arm IK is solved

IK runs entirely inside `ik_node` (package `kinematics`), which `vr_teleop` feeds through the `/left/pose` and `/right/pose` topics. One `_ArmContext` (wrapping one `ArmIKSolver`) is created per arm, each loading Daksha's full-body URDF (`daksha_description_full_body/urdf/robot.urdf`) and solving for its own 7 joints (`left_joint_1..7` / `right_joint_1..7`) against a dedicated TCP frame (`left_tcp` / `right_tcp`). The underlying solver is **Placo** (`placo.KinematicsSolver`), a whole-body task-based kinematics library; the frame task is configured as a soft constraint with weight 1.0 so it always has priority over the softer posture task described below.

5.1 Enable gate

Each arm only reacts to pose updates while its controller's grip button is held: `joy_cb` sets `ctx.enabled` from `axes[3] > 0.5` on the corresponding `/quest/{arm}/joy` message. When disabled, incoming poses on `pose_cb` are dropped outright ("STOP IK") — this is the clutch that lets the operator reposition the physical controller without dragging the arm.

5.2 Orientation correction

The controller's raw orientation is not used directly. `correct_orientation()` decomposes it into yaw/pitch/roll (optionally swapping roll and pitch via the `swap_roll_pitch` parameter), applies a per-arm direction multiplier and a fixed degree offset (independently tuned for left vs. right in `ARM_ORIENTATION_DEFAULTS`), then multiplies on two more fixed quaternions: a Quest→ROS axis correction (+90° about X, then +180° about Z) and a fixed tool-frame offset. This is what makes "point the controller forward" line up with "point the gripper forward" for each arm's particular mounting.

5.3 Smoothing filters

Before solving, `ArmIKSolver.process()` filters both position and orientation to remove headset jitter and avoid step changes at the enable gate:

- **Position:** exponential moving average, $\text{filt_pos} = (1-\alpha) \cdot \text{filt_pos} + \alpha \cdot \text{raw}$ with $\alpha_{\text{pos}} = 0.8$.
- **Orientation:** quaternion SLERP toward the new target with $\alpha_{\text{rot}} = 0.3$, after sign-matching the new quaternion to the previous one so the interpolation always takes the short way round.

The filtered orientation is then applied relative to a fixed reference rotation (`ref_rot`, the arm's orientation at the zero pose) rather than used as an absolute world orientation.

5.4 The solve itself

`solve_ik(T)` runs Placo's soft frame-task solve 5 times in a row from the previous joint solution, each iteration re-seeding from the last result to help convergence, then measures the resulting Cartesian position error by running forward kinematics on the result. If that error exceeds `error_threshold` (0.05 m — typically a target that has jumped into a difficult or near-singular configuration), `solve_escape(T)` takes over: it tries 20 random joint perturbations of $\pm 60^\circ$ as alternate starting points, resolves 10 IK iterations from each, and keeps whichever converges closest to the target. This random-restart fallback is what lets the arm find its way out of local minima instead of getting stuck.

5.5 Posture bias (redundancy resolution)

A 7-DOF arm has one redundant degree of freedom for a 6-DOF end-effector pose, so infinitely many joint configurations reach the same target. A secondary `add_joints_task()` biases the solver toward a neutral, human-like configuration (default: all zeros) with a small weight (0.01) relative to the end-effector task's weight of 1.0, so it only ever influences the arm within the null space — it never fights the actual target. Toggleable at runtime via the `posture_enabled` parameter on `/dual_arm_ik` (surfaced as a switch in `vr_management_ui`); disabling it sets the posture task's weight to 0.

5.6 Self-collision avoidance

An optional Placo constraint (`add_avoid_self_collisions_constraint`) keeps configured link pairs apart by a margin (default 0.02 m), starting to act once they come within a trigger distance (default 0.05 m). It needs two pre-generated artifacts built from the *same* geometry: `robot_collision.urdf` (simplified primitive collision shapes, from `gen_collision_primitives.py`) and `collision_pairs.json` (an *allowlist* of pairs to check, from `gen_collision_pairs.py` — not an exclusion list, since most of the model's ~268 pairs are adjacent links that always overlap at their shared joint). Using primitive shapes instead of the full visual meshes is what keeps this affordable: fitting the actual meshes costs about 133 ms per solve (~8 Hz), while primitives bring it under a millisecond. If either generated file is missing at startup, the node logs an error and runs without collision avoidance rather than silently falling back to the slow mesh path. Because each arm's solver only ever updates its own joints in its own copy of the full-body model, `_mirror_other_arms()` copies the *other* arm's current joint values into each solver's model before every solve — otherwise a solver would only ever see the opposite arm frozen at its URDF default pose, which is worse than no collision checking at all because it looks protective while actually avoiding a phantom.

5.7 Velocity / acceleration limiting

Before being published, the raw IK solution is smoothed once more in joint space: `vel_acc_limit()` clamps the per-step joint delta to `max_vel` ($10^\circ/\text{step}$) and the change in that delta to `max_acc` ($2^\circ/\text{step}^2$), uniformly scaling all joints together so the arm's shape isn't distorted mid-limit. The result becomes the new `self.q` and is what actually gets published on `/joint_cmd`.

Downstream, `joint_command_limiter` (gen2 package) applies its own independent, operator-configurable velocity/acceleration limits (`max_velocity` / `max_acceleration`, adjustable live from `vr_management_ui`) before issuing the final trajectory goals — the limiting inside `ik_node` is a first, fixed safety net specifically for the teleop path.

6. How the gripper works

Daksha's IK config sets `gripper_joint=None` for both arms, so `ik_node`'s own per-arm gripper value (smoothed from the trigger inside `joy_cb`) is *computed but never published* for this robot — that hook exists for other robot configs that fold the gripper into the same `JointState` as the arm. On Daksha, gripper control instead runs entirely through the separate `vr_gripper_ctrl` node (class `VRGripperServo`), independent of the IK solve.

6.1 Reading the trigger

`vr_gripper_ctrl` subscribes to `/quest/left/joy` and `/quest/right/joy` and reads `axes[2]`, the analog trigger. The Quest reports this axis inverted (1.0 released, 0.0 fully pressed), so it is flipped to `trigger = 1 - axes[2]` and clamped to `[0, 1]` before use; changes smaller than a 0.01 deadband are ignored to suppress sensor noise.

6.2 Position servo, at 100 Hz

A fixed-rate control loop (`CONTROL_HZ = 100`) maps the trigger linearly onto a target position between closed and open, then drives the current position toward it as a velocity-limited proportional servo rather than jumping straight there:

```
target = CLOSED + trigger * (OPEN - CLOSED)
vel     = clamp(Kp * (target - current), -Vmax, +Vmax)
current ← clamp(current + vel * dt)
```

with $K_p = 10.0$ and $V_{max} = 0.10$ m/s. This is what gives the gripper a smooth, bounded-speed close/open feel instead of snapping directly to the trigger position every 10 ms.

6.3 Per-gripper sign conventions

The two grippers are mirror images in the URDF and use opposite joint sign conventions, so a single shared “open” value would drive one of them past its limit:

Gripper	Commanded joint	Closed	Open
Left	<code>left_gripper_left_joint</code>	0.0	-0.044
Right	<code>right_gripper_right_joint</code>	0.0	+0.044

Only one joint per gripper is ever published here. Each gripper's second finger (`left_gripper_right_joint` / `right_gripper_left_joint`) is declared as a URDF **mimic** joint that follows the commanded one automatically — commanding it independently would be redundant and is exactly what `joint_command_limiter`'s `MIMIC_JOINT_NAMES` set exists to filter back out if it ever appeared.

6.4 Publish gate

The control loop always runs and keeps `left_q/right_q` up to date internally, but only *publishes* the resulting `JointState` onto `/joint_cmd` while the `publish_enabled` parameter is true (default: **false**). This is the “Gripper Publish” toggle in `vr_management_ui`, on the `/vr_gripper_servo` node — a deliberate safety default so the grippers don't start moving the instant the teleop stack comes up, before an operator has confirmed the rest of the system is behaving.

From there, both the arm joint targets (from `ik_node`) and the gripper joint targets (from `vr_gripper_ctrl`) land on the same `/joint_cmd` topic, where `joint_command_limiter` (gen2 package) picks them apart by joint name,

applies its own velocity/acceleration limiting, and issues the actual JointTrajectory goals — arms to their per-arm controllers, grippers to /left_gripper_mit_controller/joint_trajectory and /right_gripper_mit_controller/joint_trajectory.

7. Supervision and safety controls

`vr_management_ui` is the operational front door to this whole pipeline. Setting its `enabled` parameter `true` starts, in order: `default_server_endpoint`, `quest_tf_switch`, `quest_tf_to_pose`, and — 5 seconds later, so the TCP/TF layer is already up — `ik_node` (with both arms enabled). Setting it `false` tears the same group down, escalating from `SIGTERM` to `SIGKILL` if a process doesn't exit within 5 seconds.

It also relays runtime parameters to the nodes it started, over standard `SetParameters/GetParameters` service calls, and exposes them as one-click toggles on its web page at `http://<host>:8101`:

Control	Target parameter	Effect
Mode (Normal/Teach)	<code>/mode_toggler mode</code>	Switches the robot between normal operation and teach mode.
Posture	<code>/dual_arm_ik posture_enabled</code>	Enables/disables the human-like posture bias (§5.5).
Collision	<code>/dual_arm_ik collision_enabled</code>	Enables/disables self-collision avoidance (§5.6); fails if the pair allowlist never loaded.
Gripper Publish	<code>/vr_gripper_servo publish_enabled</code>	Arms/disarms gripper commands onto <code>/joint_cmd</code> (§6.4).
Max Vel / Max Acc	<code>/joint_command_limiter max_velocity / max_acceleration</code>	Live-adjusts the downstream trajectory limiter's caps.
E-STOP	HTTP POST to a separate <code>:8100</code> service	Kills the entire ROS 2 bringup, not just the VR stack — the panic button.

The same page streams live stdout from the TCP bridge and from `ik_node` in two side-by-side panels, which is normally the fastest way to see why a pose or IK update stopped arriving.

8. Launch files

Launch file	Starts	Use when
<code>vr_control_switch.py</code>	<code>endpoint + quest_tf_switch + quest_tf_to_pose</code>	Default choice. Normal or mirrored, switchable at runtime.
<code>vr_control.py</code>	<code>endpoint + quest_tf + quest_tf_to_pose</code>	Fixed normal mapping only.
<code>vr_control_mirror.py</code>	<code>endpoint + quest_tf_mirror + quest_tf_to_pose</code>	Fixed mirrored mapping only.
<code>endpoint.py</code>	<code>endpoint only</code>	Debugging the raw <code>/quest/*</code> topics, or running a TF node yourself.

Typical bring-up:

```
ros2 launch vr_teleop vr_control_switch.py
ros2 run kinematics ik_node --ros-args -p arms:=['left','right']"
```

(`vr_management_ui`, if enabled, starts both of these for you.)

9. Troubleshooting quick reference

Symptom	Likely cause / fix
No /quest/* topics	Headset not connected — check the IP entered in the Unity app matches hostname -I, both devices share a subnet, and port 10000 is open.
Topics publish but no TF appears	No TF node running — endpoint.py alone starts no TF broadcaster; use vr_control_switch.py or run quest_tf_switch alongside it.
TF_REPEATED_DATA / flickering TF	The TF nodes pass the incoming header stamp through unchanged; a Unity side sending zero or non-monotonic timestamps will be rejected by TF.
Left/right controllers feel swapped	That's the difference between normal and mirrored mode — flip it with the set_mirror service.
Arm won't move even with grip held	Check axes[3] on /quest/{arm}/joy actually crosses 0.5, and that /left,right/pose is arriving (quest_tf_to_pose logs once it starts looking transforms up).
Gripper doesn't move	Most common cause: publish_enabled on /vr_gripper_servo is still false (its default) — flip “Gripper Publish” in vr_management_ui, or set the parameter directly.
Arm visibly lags around ~8 Hz	Collision avoidance fell back to full mesh geometry, or the primitive artifacts are stale/missing — regenerate with gen_collision_primitives.py and gen_collision_pairs.py from the current URDF.